

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – EXPERIMENTS**

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	C Shahanaj	Reg. No.:	192372292
Section / Batch:	CSE AI/ 2023	Date:	4/08/2026
Instructions to Students • Answer the following question. Total Marks: 100. • Write the algorithm and execute the code for the given experiment. • Follow a well-structured, systematic approach to design and implement an optimal solution. • Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.			

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I _____shahanaj/192372292_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __shahanaj_____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				

Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____ _____	Date: _____ _____	Date: _____ _____

CODE FOR MERGE SORT:

```
#include <stdio.h>
```

```
// Function to merge two sorted subarrays
```

```
void merge(int arr[], int left, int mid, int right)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    // Copy data to temporary arrays
```

```
    for (i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];
```

```
    for (j = 0; j < n2; j++)
```

```
R[j] = arr[mid + 1 + j];
```

```
i = 0;
```

```
j = 0;
```

```
k = left;
```

```
// Merge the temporary arrays
```

```
while (i < n1 && j < n2)
```

```
{
```

```
    if (L[i] <= R[j])
```

```
    {
```

```
        arr[k] = L[i];
```

```
        i++;
```

```
    }
```

```
    else
```

```
    {
```

```
        arr[k] = R[j];
```

```
        j++;
```

```
    }
```

```
    k++;
```

```
}
```

```
// Copy remaining elements of L[]
```

```
while (i < n1)
```

```
{
```

```
    arr[k] = L[i];
```

```
    i++;
```

```
    k++;
```

```
}
```

```
// Copy remaining elements of R[]
```

```
while (j < n2)
```

```
{
```

```
    arr[k] = R[j];
```

```
    j++;
```

```
    k++;
```

```
}
```

```
}
```

```
// Function to perform Merge Sort
```

```
void mergeSort(int arr[], int left, int right)
```

```
{
```

```
    if (left < right)
```

```
    {
```

```
        int mid = (left + right) / 2;
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }
```

```
}
```

```
// Main function
```

```
int main()
```

```
{
```

```
int arr[100], n, i;

printf("Enter the number of elements: ");
scanf("%d", &n);

printf("Enter the elements:\n");
for (i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}
mergeSort(arr, 0, n - 1);

printf("Sorted array: ");
for (i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}
```

OUTPUT:

Q1

Merge Sort
10 marks

1/1 answered

Q1 Merge Sort

10 marks

Analyze Merge Sort's memory utilization by tracking auxiliary space usage during execution. Record memory consumption for different input sizes. Interpret how additional space contributes to stability. Justify trade-offs between time and space complexity.

Your Code

```
1 #include<stdio.h>
2 void merge(int a[],int l,int m,int r)
3 {
4     int i,j,k;
5     int n1=m-l+1;
6     int n2=r-m;
7     int L[n1],R[n2];
8     for (i=0;i<n1;i++)
9         L[i]=a[l+i];
10    for (j=0;j<n2;j++)
11        R[j]=a[m+1+j];
12    i=0;
13    j=0;
14    k=l;
```

Input

```
5
5 3 8 1 2
```

Output

```
Enter number of elements: Enter
elements:
Sorted array:
2 3 2 3 5
Auxiliary space = 20 bytes
```